RSpec signals failure by raising an exception. Other test frameworks use a similar technique when an assertion fails.

Wrapping Your Subject With expect

Ruby begins evaluating your expectation at the expect method. We'll start there, too. Type the following code into your IRB session:

```
>> expect_one = expect(1)
=> #<RSpec::Expectations::ExpectationTarget:0x007fb4eb83a818 @target=1>
```

Here, our subject is the number 1. We've wrapped it in the expect method to give ourselves a place to attach methods like to or not_to. In other words, the expect method wraps our object in a test-friendly adapter.

What Happened to should?

If you've used earlier versions of RSpec, you're probably familiar with the older should syntax for expectations:

```
'food'.should match(/foo/)
```

While this notation was easy to read and worked well in most cases, it had a few drawbacks. To implement it, RSpec had to monkey-patch every object in the system with should and should_not methods. This caused confusing errors in certain edge cases, such as using BasicObject or delegate to proxy method calls to an underlying object.

The new expect syntax is just as readable and is much easier to use correctly. Moreover, its implementation requires no monkey-patching at all.[a]

a. http://rspec.info/blog/2012/06/rspecs-new-expectation-syntax/

Using a Matcher

If expect wraps your object for testing, then the *matcher* actually performs the test. The matcher checks that the subject satisfies its criteria. Matchers can compare numbers, find patterns in text, examine deeply nested data structures, or perform any custom behavior you need.

The RSpec::Matchers module ships with built-in methods to create matchers. Here, we'll use its eq method to create a matcher that only matches the number 1:

```
>> be_one = eq(1)
=> #<RSpec::Matchers::BuiltIn::Eq:0x007fb4eb82dd98 @expected=1>
```

This matcher can't do anything on its own; we still need to combine it with the subject we saw in the previous section.